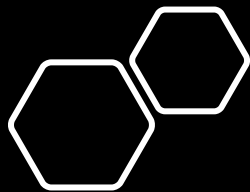


Tópicos especiais em desenvolvimento de Software

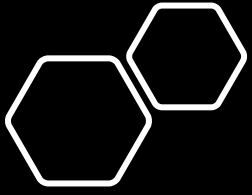
Professor: Dreslei Garbin Barcelos





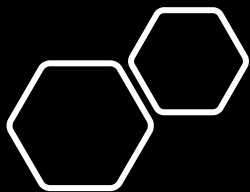
História do PHP





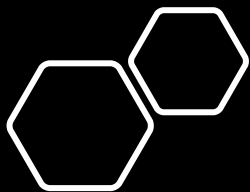
História do PHP

- Criado em 1994 por Rasmus Lerdorf, um simples conjunto de binários Common Gateway Interface (CGI) escrito em linguagem de programação C.
- Originalmente usado para acompanhamento de visitas para seu currículo online, ele nomeou o conjunto de scripts de "Personal Home Page Tools" mais frequentemente referenciado como "PHP Tools."
- Em Junho de 1995, o código fonte do PHP Tools abre para o público, o que permitiu que desenvolvedores usassem da forma como desejassem. Isso permitiu - e encorajou - usuários a fornecerem correções para bugs no código, e em geral, aperfeiçoá-lo.



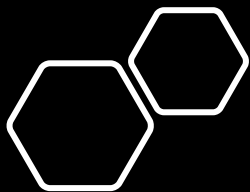
PHP 3

- Recursos para aplicações e-commerce.
- A primeira versão que se assemelha com o PHP nos dias atuais.
- PHP foi todo refeito no início do projeto da versão 3.
- O projeto foi renomeado simplesmente para 'PHP' – **PHP: Hypertext Preprocessor**.



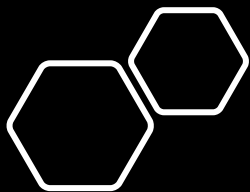
PHP 4

- Nessa versão foi reescrito o core do PHP.
- Melhorar performance das aplicações .
- Baseado no motor 'Zend Engine'.
- Uma variedades de novos recursos.



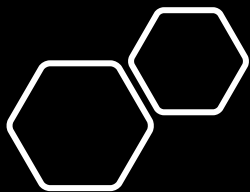
PHP 5

- Lançado em 2004.
- Zend engine 2.0, juntamente com um novo modelo de objeto.
- Adicionado biblioteca PDO, melhorando a comunicação com o BD.
- Filtro de extensões por padrão e suporte ao JSON nativo.



PHP 7

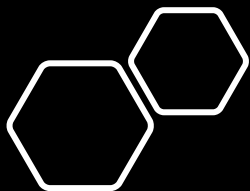
- Muito mais performance, melhorando a alocação da memória
- Zend engine 3.0
- Melhorias no tratamento dos erros fatais
- Indução de tipos em variáveis e funções.



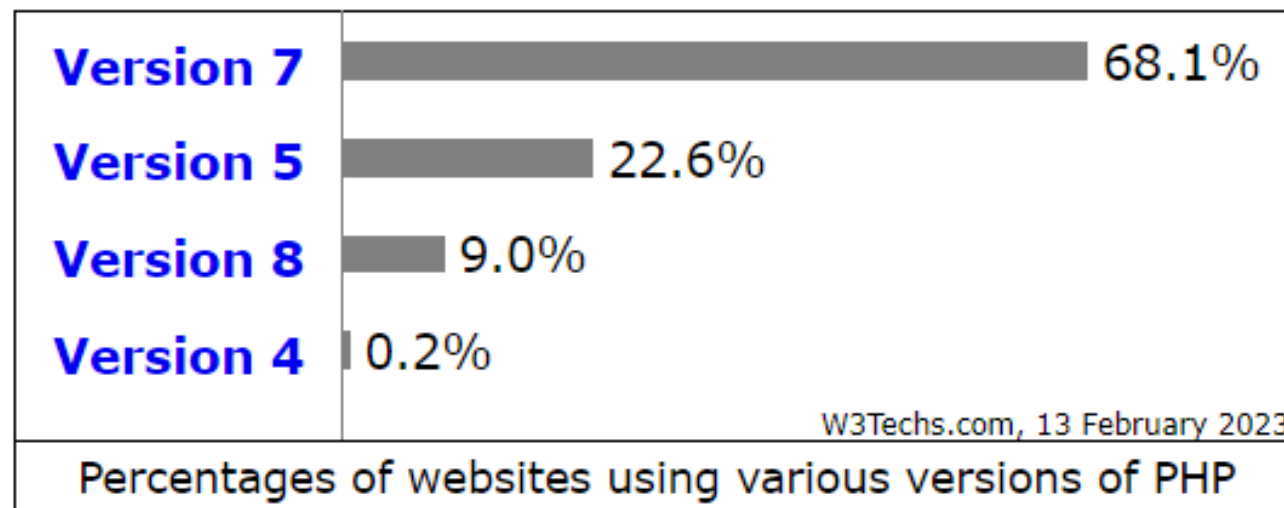
PHP 8

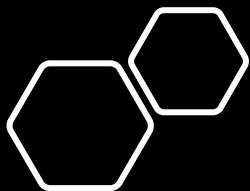
- Lançamento 2020.
- Compilador JIT (Just in Time)
- Arquivos de erros mais elaborados e precisos.
- Novas funções e mudanças no método Construct.



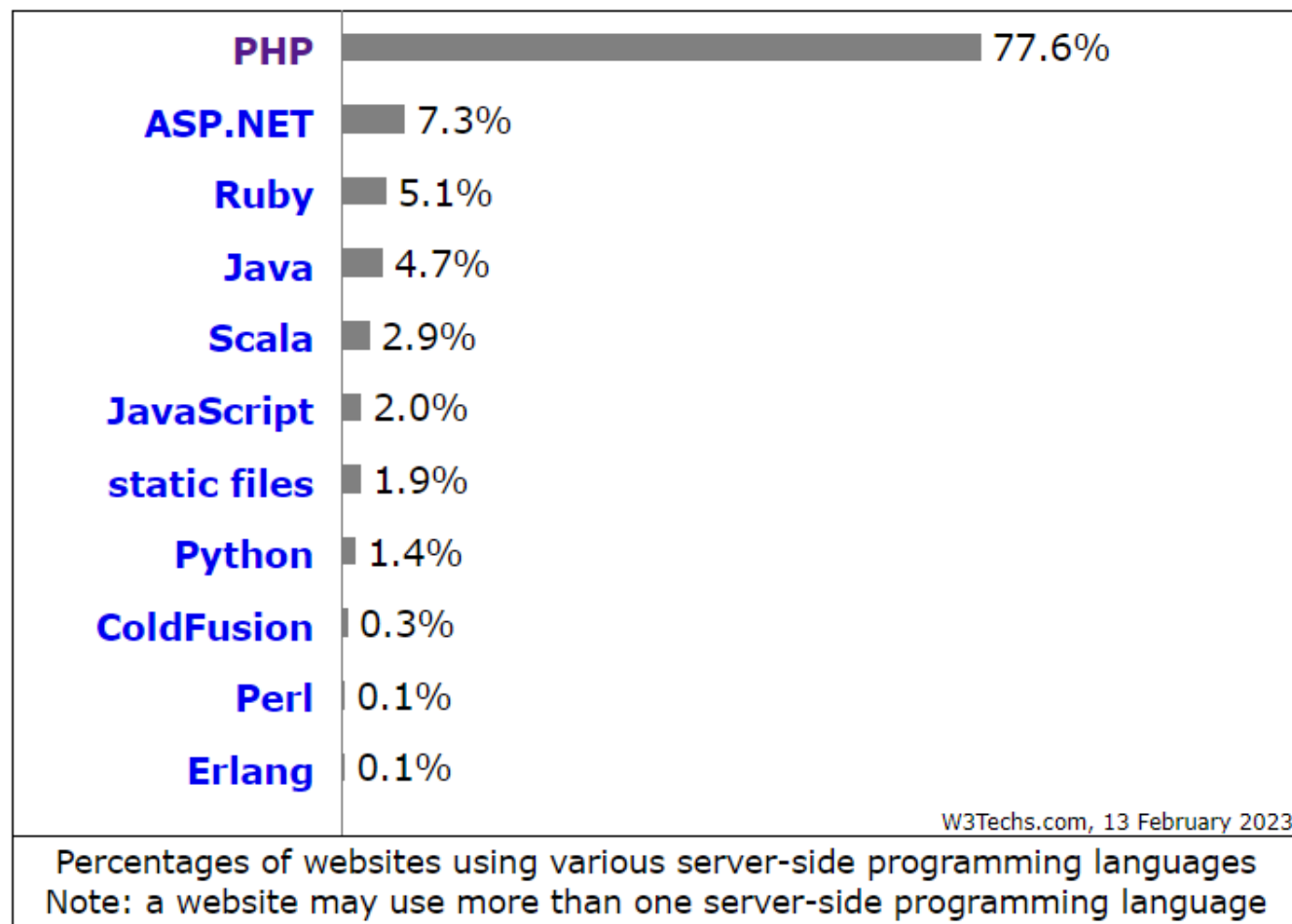


Versões mais usadas



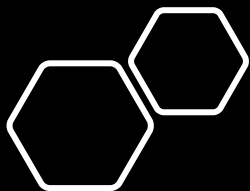


Sites que usam
linguagens
server-side.



Sites que usam linguagens server-side

[illegible]

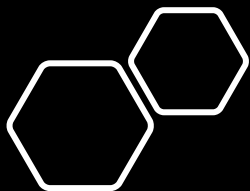


O que é PHP

É um subconjunto de linguagens de scripts como JS e Python. A diferença é que PHP é mais usado para comunicação do lado do servidor.

Serve para automatizar a execução de tarefas.

Pode ser integrado em um arquivo HTML, isso tornou a linguagem sendo mais usada para o desenvolvimento web.



Benefícios

Fácil de aprender — Uma ótima documentação

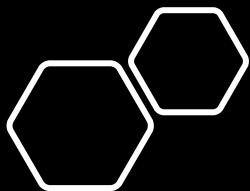
Amplamente utilizada — ela é utilizada para criar qualquer tipo de plataformas como e-commerce, blogs, redes sociais e assim em diante.

Baixo custo — ela é de código-aberto então você pode usá-la gratuitamente.

Grande comunidade — se você encontrar qualquer problema com ela, não é necessário se preocupar porque há muitos blogs sobre PHP na internet.

Integração com bases de dados — Como MySQL, ORACLE, DB2 Sybase etc...





Variáveis

\$texto declarada como uma string.

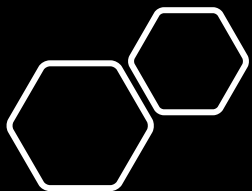
\$numero declarada como um inteiro.

echo imprimir o conteúdo dessa variável.

```
<?php
    $texto = "Aula 01 - Php";
    $numero = 8;

    echo $texto; // Aula 01 - Php
    echo $numero; // 8

    <?
    // Tag curta
    ?>
?>
```



Variáveis

A variável é determinada dinamicamente com base no valor atribuído a ela.

Não inicie o nome de uma variável com números.

Não utilize espaços em brancos.

Não utilize caracteres especiais, somente underscore (underline) .

Crie variáveis com nomes que sejam claros e de fácil entendimento.

Evite utilizar letras maiúsculas.

Tipos

Não é preciso especificar os tipos de variáveis !!!



Strings: é uma palavra ou frase entre aspas simples ou duplas.



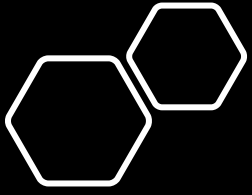
Integer: é um número inteiro, podendo ser negativo ou positivo.



Float: armazena números com ponto flutuante (reais).



Booleanos: só pode expressar dois valores: TRUE (1) ou FALSE (0);

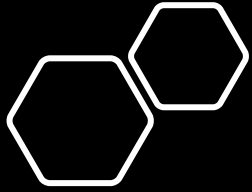


Variáveis

Quando é necessário interpretar o valor da variável dentro de uma **string** é obrigatório o uso de aspas duplas.

```
<?php
    $aula = "Aula 01 - Php";

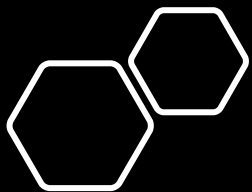
    echo  = "Aula de php, $aula";
    echo  = 'Aula de php, $aula';
?>
```



Operadores Aritméticos

- Adição: +
- Subtração: -
- Multiplicação: *
- Divisão: /
- Módulo: %

```
<?php
    $valor1 = 10;
    $valor2 = 15;
    $valor3 = $valor1 * $valor2; // saída é 150
    $valor4 = $valor1 + $valor2; // saída é 25
    $valor5 = $valor3 - $valor4; // saída é 125
?>
```

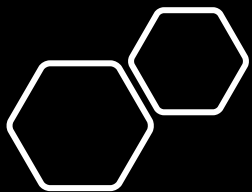


Tipos – Variáveis

Os tipos de variáveis em PHP são dinâmicos.

Reconhecendo que há um número entre as **aspas**.

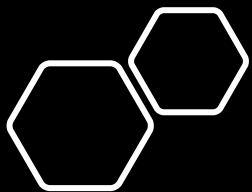
```
<?php
    $nota1 = "125"; // string
    echo $a + 25; // 150, int
    echo $a + '5 ustart'; // 10, int
?>
```



Concatenação

Usa-se um **ponto** para concatenar strings.

```
<?php
    echo "Aulas," . "UPF";
    // Aulas, UPF
?>
```

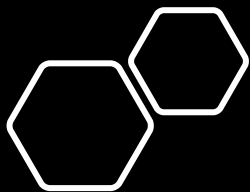


Operadores de Atribuição

Para definir variáveis com os seus valores

```
<?php
$valor1 = 22; // A variável $valor1 é igual a 22
$valor1 += 5; // Somamos 5 ao valor da $valor1;
echo $valor1; // 27

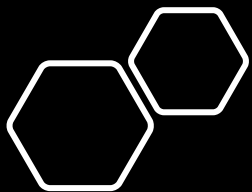
$valor2 -= 5; // Subtraímos 5 ao valor da variável $valor2;
$valor2 *= 5; // Multiplicamos o valor da variável $valor2 por 5;
$valor2 /= 5; // Dividimos o valor da variável $valor2 por 5.
?>
```



Operadores de incremento

Incrementar ou decrementar as variáveis

```
<?php
    $u = 15;
    echo --$u; // Decrementamos 15 e retornamos o valor
    echo $u--; // Retornamos o valor e decrementamos 15
    echo ++$u; // Incrementamos 15 e retornamos o valor
    echo $u++; // Retornamos o valor e incrementamos 15
?>
```

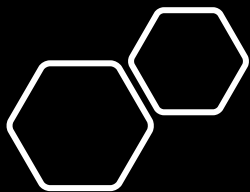


Operadores Relacionais

Comparar valores

- Diferente: **!=** ou **<>**
- Menor que: **<**
- Maior que: **>**
- Menor ou igual: **<=**
- Maior ou igual: **>=**

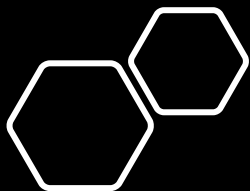
- Igual: **==** *Compara apenas os valores*
- Idêntico: **===** *Compara valor e o seu tipo*



Operadores Lógicos

Combinar expressões lógicas

- \$a **and** \$b: quando A e B forem verdadeiros.
- \$a **or** \$b: quando A ou B forem verdadeiros.
- \$a **xor** \$b: quando A ou B forem verdadeiros, mas não os dois.
- **!**\$a: verdadeiro se A for falso.
- \$a **&&** \$b: quando A e B forem verdadeiros.
- \$a **||** \$b: quando A ou B forem verdadeiros.



Estrutura de Decisão

\$idade = variável que armazena um numero inteiro.

IF = verifica se a \$idade é menor que 18

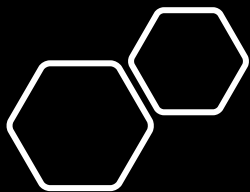
ELSE = Oposto da primeira condição

IF -> Se

ELSE -> Senão

```
<?php
    $idade = 15;

    if($idade < 18) {
        echo 'Não pode acessar esse site!';
    } else {
        echo 'Seja muito bem vindo';
    }
?>
```

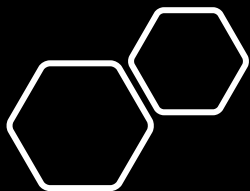


Estrutura de Decisão

Permite que o programa selecione diferentes ações, executadas com base em condições específicas.

```
<?php
    $idade = 36;
    $documento = true;

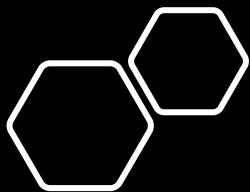
    if ($idade >= 18 && $documento == true) {
        echo 'Seja muito bem vindo(a)';
    } else {
        echo 'Você não tem idade suficiente ou não apresentou seu documento';
    }
?>
```



Switch/case

- Quando existe múltiplas opções e queremos realizar uma ação diferente para cada uma delas.
- Avalia apenas condições de igualdade.
- Sintaxe mais enxuta.

```
switch ($mes) {  
    case '01':  
        $mes_extenso = 'JAN';  
        break;  
    case '02':  
        $mes_extenso = 'FEV';  
        break;  
    case '03':  
        $mes_extenso = 'MAR';  
        break;  
    default:  
        $mes_extenso = "Este mês não foi encontrado";  
}  
return $mes_extenso;
```



Métodos: \$_POST e \$_GET

\$_GET = Coletar dados enviados pela URL ,
menos seguro, utilizado em pesquisas e rotas.

\$_POST = Coletar dados enviados pelo método
http , + seguro, envio de informações ilimitadas.

- Está é uma variável **superglobal** disponível em todos os escopos.
- Envia dados a serem processados.
- Solicitar que o servidor web aceite os dados.

\$_POST

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <title>Atividade_01</title>
    <meta charset="utf-8"/>
  </head>
  <body>
    <form action="atividade1.php" method="post">
      <p>
        Insira um Valor: <input type="text" name="valor">
      </p>
      <p>
        <input type="submit" value="Enviar">
      </p>
    </form>
  </body>
</html>
```

\$_POST

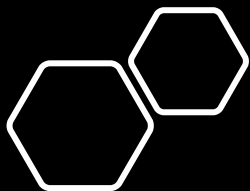
```
<?php

$valor1 = $_POST['valor1'];
$valor2 = $_POST['valor2'];

$resultado = $valor1 + $valor2;

if ($resultado >= 50) {
    echo "O valor inserido eh maior ou igual a 50!";
} else {
    echo "O valor inserido eh menor que 50!";
}

?>
```



While

`$x = 1;` Inicializa o contador do loop (`$x`), e coloca como valor inicial: 1

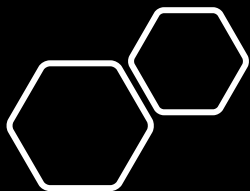
`$x <= 5` Continua o loop enquanto `$x` for menor ou igual a 5

`$x++;` Acrescenta ao contador do loop o valor +1 a cada interação

Repete o processo até a condição ser **false**

```
while (condition is true) {  
    código a ser executado;  
}
```

```
$x = 1;  
  
while($x <= 5) {  
    echo "O número é: $x <br>";  
  
    $x++;  
}
```



Do While

O laço de repetição sempre executará o bloco de código uma vez

Verifica a condição e repetirá o loop enquanto a condição especificada for verdadeira.

```
do {  
    código a ser executado;  
} while (condition is true);
```

```
<?php  
  
$x = 1;  
  
do {  
    echo "O número é: $x <br>";  
    $x++;  
} while ($x <= 5);  
  
?>
```

The Learning Pyramid

